

Lab 2 Interacting with the Olofab Question API

Overall Objectives:

- Students will practice making `GET`, `POST`, `PUT`, and `DELETE` HTTP requests using Python's `requests` library.
- Students will learn to construct JSON payloads for `POST` and `PUT` requests according to the specified API schema.
- Students will handle API responses, checking status codes and parsing JSON data.
- Students will gain experience interacting with a multi-endpoint API.

API Base URL: `https://presqa.olofab.se/api`

API Documentation: `https://presqa.olofab.se/api/docs` (Students should open and refer to this!)

Prerequisites:

- VS Code with Dev Container (Python 3) set up from the morning session.
- The `requests` library installed in the Dev Container (`pip install requests`).
- Basic understanding of Python syntax, variables, and dictionaries from the morning lab.
- Internet access from the Dev Container.

Instructions:

Part 0: Setup and API Exploration (10 minutes)

1. Open Your Project & Create a New Script:

- Ensure you are in your VS Code window, connected to the "Dev Container: Python 3" for your `Day2_Python_Labs` project.
- In the VS Code Explorer, create a new Python file named `olofab_api_client.py`.
- Add the import statement at the top:

```
import requests
import json # For pretty printing JSON later
```

2. Define Base URL:

- `BASE_URL = "https://presqa.olofab.se/api"`

3. Explore the API Documentation:

- Open `https://presqa.olofab.se/api/docs` in your web browser.
- Crucially, examine the schemas for:
 - `Question` (the response model when you GET a question)
- Use the "Try it out"-button to interact with the API.

Part 1: Reading All Questions (GET Request)

Fetch All Questions:

Add the following code to `olofab_api_client.py` :

```
def get_all_questions():
    print("\n Fetching all questions ")
    url = f"{BASE_URL}/question"
    try:
        response = requests.get(url)
        response.raise_for_status()

        questions = response.json()
        print(f"Successfully fetched {len(questions)} questions.")

        if questions:
            print("First question details:")
            print(json.dumps(questions[0], indent=2))
            return questions
    except requests.exceptions.HTTPError as http_err:
        print(f"HTTP error occurred: {http_err} - {response.text}")
    except Exception as err:
        print(f"Other error occurred: {err}")
    return None

# Call the function
all_questions = get_all_questions()
```

2. Run Your Script:

- Open the integrated terminal in VS Code.
- Run: `python olofab_api_client.py`
- Observe the output.

Part 2: Creating a New Question (POST Request)

Define a Function to Create a Question. Add this function to your script:

```
def create_question(question_text, answer_text):
    print(f"\n Creating a new question: '{question_text}' ")
    url = f"{BASE_URL}/question"
```

```

payload = {
    "question": question_text,
    "answer": answer_text
}

try:
    response = requests.post(url, json=payload)
    response.raise_for_status()

    created_question = response.json()
    print("Successfully created question:")
    print(json.dumps(created_question, indent=2))
    return created_question # This will include the 'id'
except requests.exceptions.HTTPError as http_err:
    print(f"HTTP error occurred: {http_err} - {response.text}")
except Exception as err:
    print(f"Other error occurred: {err}")
return None

# Call the function to create a new question
new_q_text = "What is the capital of Sweden?"
new_q_answer = "Stockholm"
created_q_object = create_question(new_q_text, new_q_answer)

created_question_id = None
if created_q_object:
    created_question_id = created_q_object.get("id")
    print(f"ID of our new question: {created_question_id}")

```

2. Run Your Script:

- Execute `python olofab_api_client.py`.
- You should see the details of the newly created question, including its server-assigned `id`.
- Verify: You can call `get_all_questions()` again to see your new question.

Part 3: Reading a Specific Question (GET with Path Parameter)

Define a Function to Get a Specific Question. (This function remains largely the same as before, as it just uses the ID)

```

def get_specific_question(question_id):
    if not question_id:
        print("No question ID provided to fetch.")
        return None
    print(f"\n Fetching question with ID: {question_id} ")

```

```

url = f"{BASE_URL}/question/{question_id}"
try:
    response = requests.get(url)
    response.raise_for_status()

    question = response.json()
    print("Successfully fetched specific question:")
    print(json.dumps(question, indent=2))
    return question
except requests.exceptions.HTTPError as http_err:
    print(f"HTTP error occurred: {http_err} - {response.text}")
except Exception as err:
    print(f"Other error occurred: {err}")
return None

# Call the function using the ID from the question you created
if created_question_id:
    get_specific_question(created_question_id)
else:
    print("Skipping get_specific_question as no question was created successfully.")

```

Run Your Script.

Part 4: Updating a Question (PUT Request) (20 minutes)

Define a Function to Update a Question:

```

def update_question(question_id, new_question_text=None, new_answer_text=None):
    if not question_id:
        print("No question ID provided to update.")
        return None

    print(f"\n Updating question ID {question_id} ")
    url = f"{BASE_URL}/question/{question_id}"

    payload = {}
    if new_question_text is not None:
        payload["question"] = new_question_text
        print(f"Setting new question text: '{new_question_text}'")
    if new_answer_text is not None:
        payload["answer"] = new_answer_text
        print(f"Setting new answer text: '{new_answer_text}'")

    if not payload:
        print("No new data provided for update.")
        return None # Or fetch and return the existing if no change

    try:
        response = requests.put(url, json=payload)

```

```

        response.raise_for_status()

        updated_question = response.json()
        print("Successfully updated question:")
        print(json.dumps(updated_question, indent=2))
        return updated_question
    except requests.exceptions.HTTPError as http_err:
        print(f"HTTP error occurred: {http_err} - {response.text}")
    except Exception as err:
        print(f"Other error occurred: {err}")
    return None

# Call the function to update the question
if created_question_id:
    updated_q_text = "What is the main programming language used in FastAPI?"
    updated_q_answer = "Python"
    update_question(created_question_id, new_question_text=updated_q_text,
new_answer_text=updated_q_answer)
    # Optionally, call get_specific_question again to see the update
    get_specific_question(created_question_id)
else:
    print("Skipping update_question as no question was created successfully.")

```

Run Your Script.

Part 5: Deleting a Question (DELETE Request) (15 minutes)

Define a Function to Delete a Question:

```

def delete_question(question_id):
    if not question_id:
        print("No question ID provided to delete.")
        return None
    print(f"\n Deleting question with ID: {question_id} ")
    url = f"{BASE_URL}/question/{question_id}"
    try:
        response = requests.delete(url)
        response.raise_for_status()

        # API docs state it returns the deleted object with 200 OK.
        if response.status_code == 200 and response.content: # Check content for
safety
            deleted_item_info = response.json()
            print("Successfully deleted question. API returned:")
            print(json.dumps(deleted_item_info, indent=2))
        else: # General success based on raise_for_status if not 200 with content
            print(f"Successfully initiated delete for question ID {question_id}
(Status {response.status_code}).")
    return True

```

```

except requests.exceptions.HTTPError as http_err:
    print(f"HTTP error occurred: {http_err} - {response.text}")
except Exception as err:
    print(f"Other error occurred: {err}")
return False

# Call the function to delete the question
if created_question_id:
    delete_question(created_question_id)
    # Try to fetch it again - should result in a 404 Not Found error
    print("\nAttempting to fetch the deleted question (should fail with 404):")
    get_specific_question(created_question_id)
else:
    print("Skipping delete_question as no question was created successfully.")

```

Lab Completion & Discussion Points:

- Ensure your script can perform all CRUD (Create, Read, Update, Delete) operations successfully with the correct data payloads.
- How does the `id` field behave? (Generated by server on create, used for lookups/updates/deletes).
- What happens if you try to `POST` data that doesn't match the `QuestionCreate` schema (e.g., you include an `id` or miss the `answer`)? The API should return a 422 Unprocessable Entity error. Test this!
- What happens if you try to `PUT` with an empty payload? (The API might update nothing, or it might error depending on its implementation – test it).